

DriftConditions Implementation Plan

Authentication and Authorization Upgrade

1. Purpose

The primary goal of this upgrade is to enhance the user experience (UX) by introducing OAuth 2.0 support for Google and GitHub sign-in, simplifying the login process and allowing users to authenticate via trusted external providers. This upgrade is being implemented despite the low volume of users (tens per week) and the existing security foundation (which has always used HTTP-only cookies for session tokens). The new design will unify all sign-in methods, ensuring that whether a user logs in locally or via OAuth, they are recognized as a single identity with consistent roles and permissions. While the system maintains robust security, the primary value is in improving convenience and eliminating account duplication risks. The low-stakes nature of the site, where new content requires role-based approval, mitigates the risk of a stolen access token. This document details the technical implementation, showing how backend logic, database design (user linking), and frontend flow work together.

2. Goals and Scope

2.1. Goals

1. **Introduce OAuth 2.0 support** for Google and GitHub sign-in so users can authenticate using trusted providers instead of manual passwords.
2. **Preserve all existing user accounts and roles**, linking new OAuth identities to existing profiles where emails match.
3. **Maintain consistent role-based access control (RBAC)** across every authentication method, ensuring permissions such as “Admin,” “Contributor,” and “Viewer” remain unchanged.
4. **Adopt modern authentication standards**, including improved password hashing, CSRF protection, reCAPTCHA on login, and SameSite cookie enforcement.
5. **Improve error handling and logging**, so failed sign-ins, expired tokens, and authorization denials are handled gracefully and recorded for debugging.
6. **Document validation and testing**, ensuring each login path, token renewal, and permission check is verified through test coverage and manual QA.

2.2. Why these goals matter:

They reduce the maintenance burden, eliminate repeated password recovery workflows, and improve user confidence. With OAuth 2.0, users never share credentials with DriftConditions directly. By using secure cookies, we continue to prevent common web vulnerabilities like token theft or misuse by malicious scripts, a benefit inherited from the original system.

2.3. Scope

This upgrade modifies:

- **Backend components:** authentication routes, middleware, and database relations (to include OAuth provider IDs).
- **Frontend components:** login forms, session checks, and protected-page access logic.
- **Security configuration:** JWT lifetime, cookie attributes (httpOnly, Secure, SameSite) and rate limiting on login endpoints.

The core login flow remains familiar, though all new sign-ups (local or OAuth) will be redirected to the profile edit page, and the Sign In page will now include provider buttons.

2.4. Out of Scope:

- Any visual redesign of the sign-in page.
- Passwordless or biometric login options.
- Integration with enterprise identity systems (Okta, Azure AD).
- Experimental or unapproved authentication features.

3. Inventory of the Current System

The existing DriftConditions authentication and authorization setup uses a traditional local username-and-password model. User identity is stored in the database, verified through backend logic, and validated using JSON Web Tokens (JWT). Although this design works reliably for basic login and access control, it lacks modern features such as OAuth integration, standardized session security, and unified role management across providers.

3.1. Authentication (Current Behavior)

At present, the authentication process is simple but limited.

When a user registers, they use the `/api/auth/signup` route; when they log in, they use `/api/auth/signin`. Both actions are handled by the backend, which talks directly to the MySQL database where usernames, emails, and hashed passwords are stored. When the user provides their credentials, the server checks them against the stored record. If the information is correct, the system generates a JWT that includes the user ID, username, and role. If the credentials do not match, the server returns an error response, and the user stays on the sign-in page.

The token is then sent back to the client. In the current version of the system, that token is stored inside a secure, HTTP-only cookie, which automatically expires and is renewed on the backend as necessary, to prevent exposure to JavaScript or cross-site attacks.

When the user signs out, the token is manually deleted from **secure, HTTP-only cookie**. The backend does not currently revoke tokens or track sessions, so logout only works on the client side. This design keeps the process simple but also exposes a few weaknesses, especially around token security and session control.

3.2. Authorization (Current Behavior)

Authorization determines what each user can do once they are logged in. Every user has a role such as admin, contributor, or viewer, and each role defines which parts of the system they can access.

These roles and permissions are stored in the database. When a request comes in for a protected endpoint (including all pages except for the homepage, sign-in, sign-up, no-auth, and 404 error pages), the backend checks whether the request includes a valid JWT. If the token is valid and the role in that token matches what the route requires, the request goes through. If the token is missing, expired, or carries the wrong role, the system blocks the request with a “Not Authorized” message.

The frontend uses that same information to adjust what users see. For example, if a user logs in as a viewer, they will only see read-only content and cannot reach admin-only pages or upload forms. This provides basic role-based control but depends entirely on the correctness of the token and the user’s browser state.

Once a user is authenticated, authorization determines what they can access within the system.

3.3. Observed Issues (Limitations and Improvements Needed)

While the existing setup functions, several issues make it unsuitable for long-term use:

- **No third-party login options:** Only local username and password authentication is supported. Users cannot sign in with Google or GitHub, which limits convenience and creates extra friction for new users.
- **Weak session control:** Logout simply removes the token from the **secure, HTTP-only cookie**. There is no server-side session tracking, meaning sessions can stay active if the user forgets to log out.
- **Duplicate identity risk:** Each account stands alone, so a user cannot link multiple logins (for example, the same email used for Google and a local account). This is not a problem with the current system, but we must be aware of this risk with the new system.
- **Limited visibility and error handling:** Authentication errors, expired tokens, and failed logins are not always logged or clearly reported, which makes debugging and security auditing harder.

These limitations primarily reduce user convenience and account flexibility, making the system less suitable for expansion. The upcoming upgrade resolves these issues by integrating OAuth

login options and introducing consistent role-based access control (RBAC) for every authentication method, while continuing to store tokens securely in HTTP-only cookies.

4. System Flow

This section explains how data and authentication requests move through DriftConditions from the moment a user clicks Sign In until the system verifies their permissions. Understanding this flow is important because it reveals exactly where the system makes security decisions, how information passes between the browser, backend, and external providers, and what happens when things fail. The updated flow provides a shared mental model so that developers, testers, and reviewers all understand the same step-by-step behavior.

4.1. Current Flow

4.1.1. Login

1. The user visits the sign-in page and types their email and password.
2. When they click Sign In, the frontend sends a POST /signin request to the backend containing those credentials.
3. The backend checks the users table in the MySQL database to find a matching account and verifies the password using bcrypt.
4. If the credentials are valid, the backend creates a JSON Web Token (JWT) that contains key details such as the user ID, username, and role.
5. The token is sent back to the browser, which stores it in a secure, HTTP-only cookie for the token, which automatically expires and is renewed on the backend as necessary.
6. Every later request to protected endpoints returns this token cookie from the auth check : Bearer <token> header.
7. Middleware on the server decodes the token to confirm that it is valid and that the user has the right role.
8. If everything checks out, access is granted. If not, the system replies with 401 Unauthorized or 403 Forbidden.

4.1.2. Signup

1. The user visits the registration page and enters their desired email and password.
2. Upon submission, the frontend sends a POST /signup request to the backend with these credentials.
3. The backend checks the users table in the MySQL database to ensure the email is not already registered.
4. The backend hashes the password using bcrypt for secure storage.
5. A new user record is created in the database with a default role.
6. The backend issues a JWT containing the new user's identity and role.
7. The token is returned and stored in a secure, HTTP-only cookie.

8. The user is redirected to the profile edit page to complete required fields, establishing a new active session.

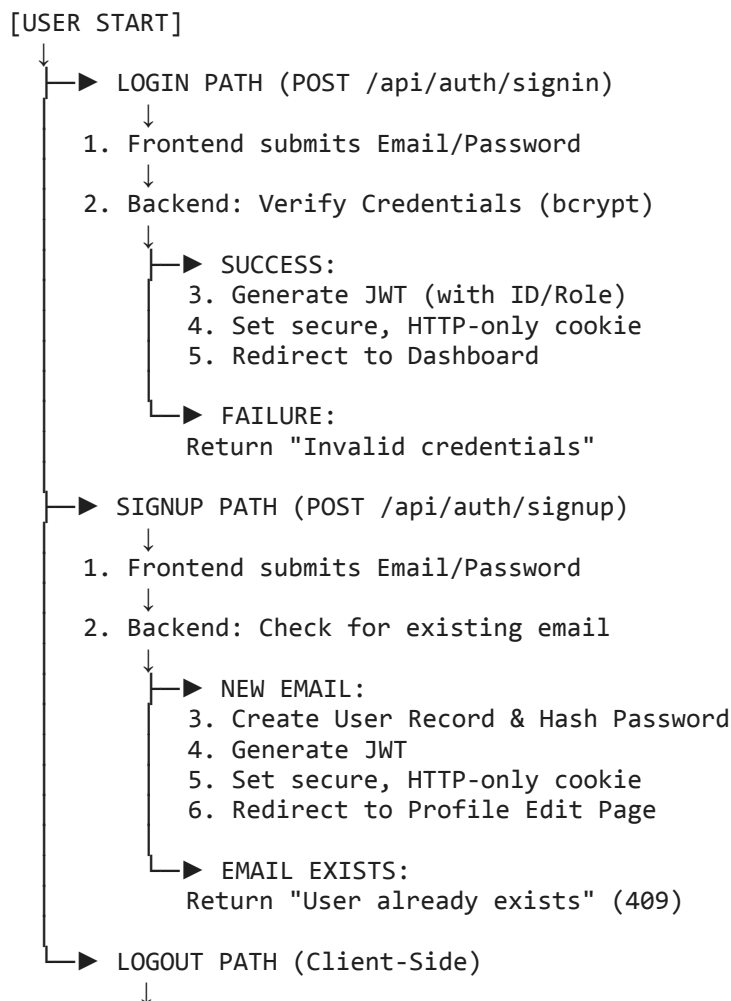
4.1.3. Logout

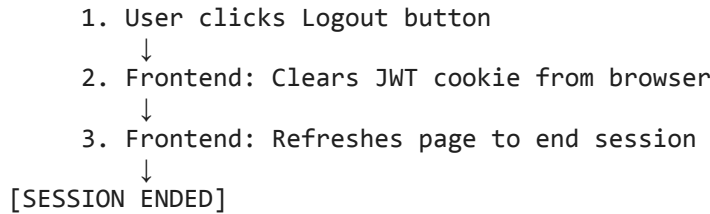
1. When the user logs out, the frontend deletes the stored token and refreshes the page, ending the session on the client side

4.1.4. Token Renewal (Sliding Session)

The system uses a Sliding Session mechanism. On every request to a protected endpoint, the `requireAuth` middleware checks the current token's expiration time. If the token is valid but falls within the final 30 minutes of its two-hour lifespan, the server automatically signs and issues a brand new JWT with a fresh two-hour TTL, overwriting the old `access_token` cookie. This happens completely transparently to the user, ensuring uninterrupted sessions and excellent UX.

4.1.5. Flow Diagram (Current System)





4.2. Proposed Flow

The new system introduces modern authentication flows using OAuth 2.0 for Google and GitHub, while still keeping local sign-in as an option.

4.2.1. Login

1. On the sign-in page, the user can now choose one of three options:
 - Local Sign In (email + password + reCAPTCHA).
 - Sign in with Google.
 - Sign in with Github.
 - Other oAuth sources may be offered in the future.
2. For local login, the browser sends a POST `/api/auth/signin` request containing the user's credentials. The backend verifies credentials and, upon success, updates the **users** table (e.g., setting **lastLoginAt** or creating a new user record) to reflect the login event.
3. For OAuth login, the frontend sends a GET `/api/auth/google` or GET `/api/auth/github` request. The backend responds by redirecting the browser to the selected provider's login page.
4. After the user successfully logs in with the provider, the provider redirects the browser back to our backend with a temporary authorization code and a state parameter for CSRF protection.
5. The backend validates the state and exchanges the authorization code with the provider for an access token.
6. Using that access token, the backend retrieves the verified profile information (email, name, provider ID).
7. The backend then checks whether this profile matches an existing user:
 - If the provider ID or email exists → link or reuse that account by inserting or updating a record in the **userIdentities** table.
8. Once the user is confirmed, the backend creates a short-lived JWT (for example, 2 hours) and sets it as an HTTP-only, Secure, SameSite=Lax cookie named `access_token`. The backend redirects the user to either the profile edit page (new users) or the dashboard (existing users).
9. All protected requests automatically include this cookie. The frontend never handles or stores the raw token.

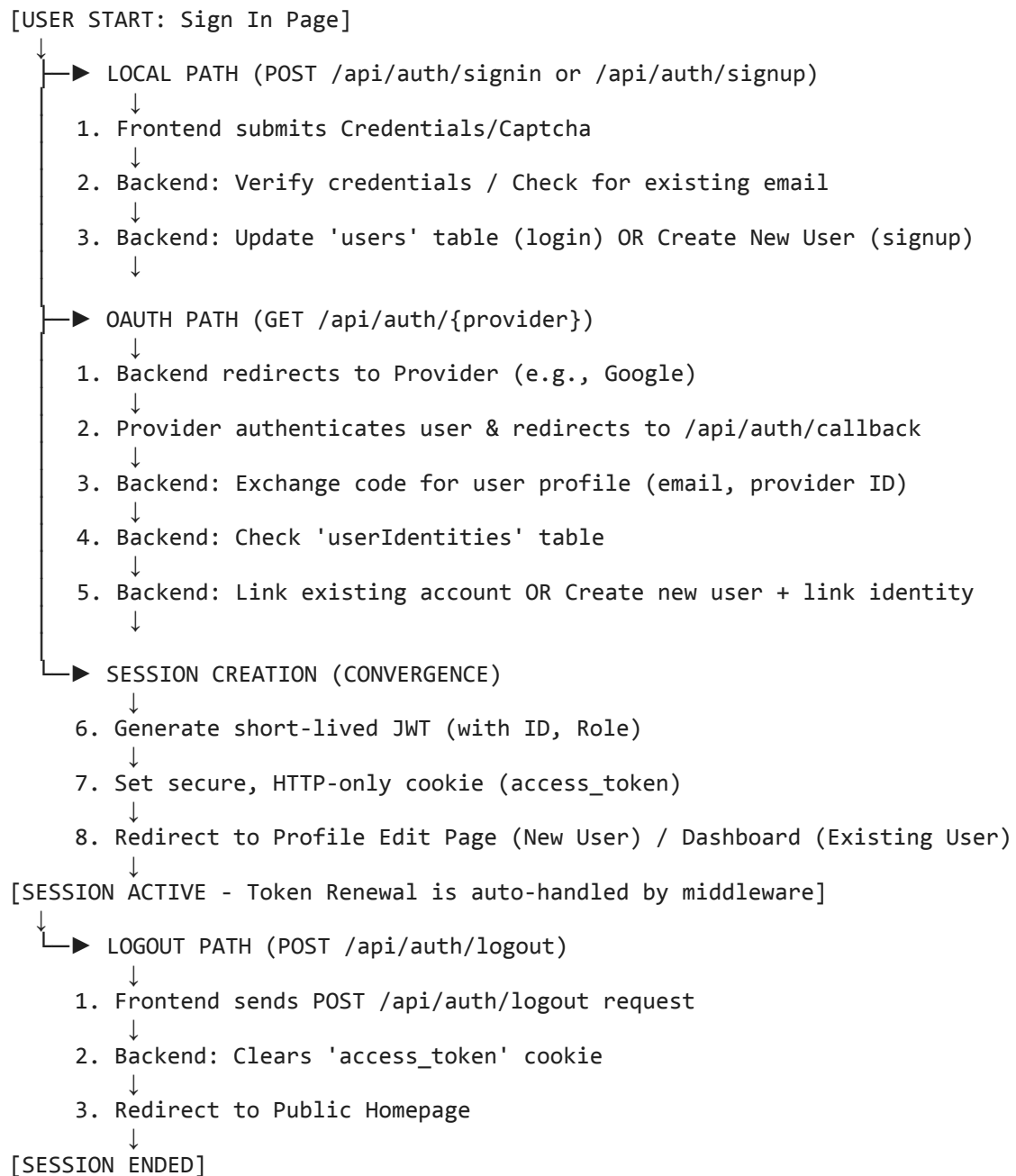
4.2.2. Signup

1. The user navigates to the dedicated Sign Up page and submits their email and password.
2. The frontend sends a `POST /api/auth/signup` request, including credentials and reCAPTCHA token.
3. The backend validates the input and confirms the email is not already in the **users** table. It then creates a new user record in the **users** table, sets the default role, signs a JWT, and sets it in a secure, HTTP-only cookie.
4. The user is redirected to the profile edit page to complete required fields, establishing a new active session.

4.2.3. Logout

1. If the user logs out, a `POST /api/auth/logout` request clears the cookie, and the browser is redirected to the public homepage.

4.2.4. Flow Diagram (Proposed System)



4.2.5. Why These Changes Matter

- It standardizes how all users — local, Google, and GitHub — are authenticated under one identity model.
- By using secure cookies, it reduces the chances of stolen sessions, simplifies logout management, and keeps security logic entirely server-side.

- The implementation uses a dedicated authentication module that allows for the easy addition of future OAuth sources, such as Apple.
- These changes make DriftConditions easier to maintain and safer for end users.

5. Strategic Options and Staged Approach

Upgrading DriftConditions authentication must be done carefully so that no users are locked out and the system remains stable throughout development. This section compares two possible strategies and presents a safe, phased rollout plan.

5.1. Option A : Dual-Track: Local + OAuth

5.1.1. What changes:

This approach keeps the existing local email-and-password login system while adding Google and GitHub OAuth2 login options. Both local and OAuth sign-ins will share the same secure cookie format and use the same role-based access control (RBAC) model.

5.1.2. Why it matters:

By keeping local login active, existing users can continue using their accounts without interruption. At the same time, new users can choose to log in through Google or GitHub, improving convenience and reducing password fatigue.

5.1.3. Pros

- Minimal disruption to current users.
- Easier rollback if any OAuth issue occurs.
- Supports both internal users (local) and external contributors (OAuth).
- Helps the team test both paths safely before full transition.

5.1.4. Cons

- Slightly more complex code since both methods must be maintained.
- Requires additional testing to ensure cookie handling works for both login types.
- Requires an additional database table for a one-to-many relationship between the user record and the OAuth identities.

5.1.5. A Staged Approach

The recommended strategy uses Option A, implemented in controlled phases. Each stage can be tested and verified independently before moving to the next.

Phase 1: Secure Cookies and Validation Hardening

- Add Google reCAPTCHA to stop automated sign-in attempts.
- Introduce rate limiting to prevent brute-force attacks.
- Keep existing local login and signup fully functional during this step.

Phase 2: Add Google/GitHub OAuth Integration

- Implement OAuth routes (/auth/google, /auth/github) and callback handling.
- Create user linking logic to match OAuth profiles to existing emails.
- Maintain shared session handling via secure cookies.
- Introduce a session view endpoint for debugging (/auth/check).

Phase 3: Identity Linking and Migration

- Verify that the linking logic correctly identifies and merges identities (Local + OAuth, OAuth + OAuth) without creating duplicates.
- Confirm that the new userIdentities table correctly stores provider data for existing and new users.
- Validate that all existing user roles and permissions carry over consistently regardless of the login method (Local or OAuth).
- Begin the gradual transition of the user base to the new unified model.

Phase 4: Monitoring and Gradual Rollout

- Add detailed logging and monitoring for sign-in attempts and failures.
- Enable feature flags to control rollout across environments.
- Conduct QA verification to ensure stability before retiring old code paths.
- Remove deprecated client-side token logic once the new system proves stable.

The Dual-Track (Local + OAuth) option ensures stability while moving DriftConditions toward modern authentication practices.

By breaking the work into phases, the team can test, debug, and document each improvement step-by-step without disrupting existing users.

This method offers the flexibility to adapt or roll back safely, making it the most practical path forward.

5.2. Option B: OAuth-Only

5.2.1. What changes

This option removes the local login system completely, requiring every user to authenticate only through Google or GitHub.

5.2.2. Why it matters

It removes password handling and related risks, simplifying backend security. However, it also makes the system dependent on third-party providers for every login.

5.2.3. Pros

- No password storage or reset management.
- Simplified authentication logic.

5.2.4. Cons

- Internal testing or offline login becomes impossible without provider access.
- Risk of login downtime if a provider is unavailable or API limits are hit.
- Migration complexity for users who originally registered with local accounts.

6. Implementation Narrative

This section describes, in the order of system evolution, how each component of the upgraded authentication and authorization stack behaves once implemented. It connects backend route logic, database changes, frontend handling, migration procedures, and validation methods into one operational story. Rather than listing edits, it explains the runtime behavior of the finished system—what the user, the API, and the database actually do during every stage of sign-in and authorization.

6.1. Backend Changes

The backend now unifies local and OAuth 2.0 flows under a dedicated **auth router**. All logic is expressed through secure, server-side cookies and standardized error responses.

6.1.1. Routing and Endpoints

Endpoint	Behavior
----------	----------

POST /api/auth/signup	Local registration: Normalizes email → checks DB → hashes password (bcrypt) → inserts record with default role → signs JWT → sets cookie
POST /api/auth/signin	Local login: Validates email/password → issues JWT cookie → returns user profile summary
GET /api/auth/:provider	OAuth init: Redirects browser to Google/GitHub auth page with state for CSRF protection
GET /api/auth/callback/:provider	OAuth return: Verifies state → exchanges code for provider token → fetches profile (email, id) → links or creates user → signs JWT cookie
POST /api/auth/check	Session status: Reads cookie → verifies JWT → returns <code>{isAuthorized, displayName, role, provider}</code>
POST /api/auth/logout	End session: Clears cookie → returns <code>{success:true}</code>

6.1.2. Middleware Enhancements

- `requireAuth` verifies JWT from `access_token` cookie; if invalid → 401 and cookie cleared.
- `requireRole(role)` compares `req.user.role` to allowed values → 403 if insufficient.
- Unified error objects (`INVALID_STATE`, `OAUTH_EXCHANGE_FAILED`, `EMAIL_EXISTS`, `FORBIDDEN`) ensure consistent client responses.
- **Rate Limiter** protects `/api/auth/signin`, `/api/auth/signup`, and `/api/auth/*` with per-IP thresholds.
- **Security Headers** (HSTS, no-sniff, CORS with credentials) added globally.

6.1.3. Token and Cookie Policy

JWTs contain: `sub` (user ID), `email`, `role`, `permissions`, `provider`, and `exp` (two-hour TTL). Cookies use these flags:

None

```
httpOnly = true
secure   = true
sameSite = 'Lax'
path     = '/'
domain   = '.driftconditions.app'
```

The cookie replaces client-side token storage, eliminating exposure to JavaScript and XSS.

6.2. Database Updates

The database layer now tracks both local and OAuth identities within a single users table. New columns record provider details without breaking existing records, so the system remains backward-compatible and ready for hybrid logins.

6.2.1. Schema Adjustments

SQL

```
-- Existing 'users' table update (only adding utility columns)
ALTER TABLE users
  ADD COLUMN displayName VARCHAR(255),
  ADD COLUMN lastLoginAt DATETIME;

-- New 'userIdentities' table to support multiple linked OAuth accounts
CREATE TABLE userIdentities (
  userId INT NOT NULL, -- Foreign key to the users table
  oauthProvider VARCHAR(32) NOT NULL,
  oauthId VARCHAR(191) NOT NULL,
  PRIMARY KEY (oauthProvider, oauthId),
  FOREIGN KEY (userId) REFERENCES users(userId) ON DELETE CASCADE
);
```

6.2.2. Why It Matters

- The new **userIdentities** table creates a crucial one-to-many relationship, allowing a single user to link multiple external providers (Google, GitHub) to their core user account, resolving the duplicate identity risk.
- The unique constraint on (oauthProvider, oauthId) ensures that each provider ID is globally unique.
- **displayName** preserves provider-supplied names for UI consistency.

- `lastLoginAt` enables usage analytics and anomaly detection.
- This structure ensures that account linking works as intended in the proposed flow, where existing email conflicts trigger controlled merging of identities.
- All existing data in the **users** table remains valid.

6.3. Frontend Changes

Frontend behavior shifts from token-driven logic to session hydration based on secure cookies. This simplifies client state and reduces attack surface without changing visible UX for users.

6.3.1. Key Changes

1. **Sign-In Page** adds buttons for “Sign in with Google” and “Sign in with GitHub,” beside the existing email/password form.
2. **reCAPTCHA** token appended to local sign-in requests for bot protection.
3. **Session Hydration:** on page load, `POST /api/auth/check` determines auth state by the backend reading the secure, HTTP-only cookie; although the cookie exists on the client, it is inaccessible to client JavaScript.
4. **Protected Routing:** if `/auth/check` returns 401 → redirect to `/signin`; if 403 → show “Not Authorized.”
5. **Logout** calls `POST /logout`, which clears cookie and reloads the public page.
6. **UI Adaptation:** menu and page access continue to use role-based permissions delivered by the backend.

6.3.2. Why It Matters

- No sensitive token ever appears in the DOM or console.
- Session state remains consistent across tabs and browser refreshes.
- Client logic is simpler and more secure.

6.4. Migration and Rollout

Migration must preserve all existing accounts and sessions while introducing OAuth without service interruption. The rollout proceeds in measured phases to allow testing and rollback at any step.

The generalized OAuth/OIDC authentication is built around the minimal `openid-client` module, which is protocol-focused and ensures the easy addition of future providers such as Apple with trivial changes.

6.4.1. Phased Execution

1. **Schema Deployment** – Apply database changes; verify unique email index and default values.

2. **Backend Release** – Deploy new auth router and middleware; enable both local and OAuth routes.
3. **Session Transition** – Enable secure cookie issuance; legacy clients auto-refresh tokens to new format.
4. **OAuth Activation** – Add Google and GitHub credentials; monitor callback success rates.
5. **Account Linking** – On login, merge duplicate emails between local and OAuth records (safely handled by unique constraint).
6. **Client Update** – Push new frontend build with provider buttons and no token storage.

Each phase includes automated tests and database backups before deployment.

6.5. Testing and Monitoring

Testing in this phase proves that each authentication flow—local, OAuth, and hybrid—behaves exactly as designed.

Rather than simply checking endpoints, these tests verify the *promises* the system makes: that tokens stay server-side, sessions refresh correctly, duplicates merge cleanly, and unauthorized access is rejected.

Every test case corresponds to a real path a user or attacker could take, ensuring the upgraded system is observable, predictable, and auditable in production.

6.5.1. Unit Tests

Objective	Focus	Expected Result
JWT Creation	Sign and verify JWTs with rotating secret; confirm exp claim honored	Token valid until expiry; invalid after expiry
Cookie Integrity	Set and clear <code>access_token</code> ; verify HTTP-only/Secure flags	Cookie unreadable from JavaScript; cleared on logout
Middleware Auth	<code>requireAuth</code> and <code>requireRole</code> logic	401 for missing/invalid JWT; 403 for insufficient role
Rate Limiter	Multiple POST <code>/api/auth/signin</code> requests	Blocks after threshold; returns 429
Input Validation	Malformed email or missing fields	400 with error code <code>INVALID_FIELD</code>

6.5.2. Integration Tests

TBD

6.5.3. Use Cases

Case 1: Brand New User (Local Email Signup)

1. A completely new visitor navigates to the **Sign Up page** and enters their email and chooses a password.
2. The user submits the form via the **POST /api/auth/signup** route.
3. Behind the scenes, the server checks and confirms there's no record for that email.
4. It then creates a brand-new user entry in the **users** table with their email, a hashed password, and default role "user".
5. After that, it signs a short-lived JWT, saves it in a secure HTTP-only cookie, and notes that the last login source was local.
6. The page refreshes into the dashboard.
7. The user is redirected to the profile edit page to complete required fields.

Case 2: Brand New User (OAuth Google or GitHub Signup)

8. A user who originally joined using Google now prefers to log in with an email and password.
9. They enter the same email on the login page, but the database shows no local password.
10. The system politely offers a "Set Password" option.
11. The user sets a new local password directly on the frontend form.
12. Once the password is created and hashed, the backend updates the password hash in the existing **users** record to link both the Google and local login methods.
13. The user is redirected to the dashboard, confirming a new active session.
1. Someone new decides not to type a password and instead clicks "Sign in with Google."
2. The browser jumps to Google's consent screen asking for permission to share profile information.
3. Once approved, the provider sends back a secure code.
4. Our server trades that code for a verified email and name, then creates a core user record in the **users** table and inserts the provider ID into the **userIdentities** table, setting the role to user.
5. It signs a JWT, stores it as a secure cookie, and redirects to profile edit page.
6. The user is redirected to the profile edit page to complete required fields.

Case 3: Existing OAuth User Logs in Locally (Local Login for the First Time)

1. A user who originally joined using Google now prefers to log in with an email and password.
2. They enter the same email on the login page, but the database shows no local password.
3. To prevent unauthorized access, the system requires a secure email verification step.
4. The user must click a “Set Password” request, which triggers the backend to send a secure, time-limited link to their verified email address.
5. Only after the user clicks this link to prove account ownership are they allowed to set a new local password.
6. Once the password is created and hashed, the backend updates the password hash in the existing `users` record to link both the Google and local login methods.
7. The user is redirected to the dashboard, confirming a new active session.
8. *Note: This case requires a new verification route and the ability to send email.*

Case 4: Existing Local User Tries OAuth Login (Account Linking)

1. A long-time local user decides to click “Sign in with GitHub.”
2. The OAuth provider confirms their email, which already exists in our database.
3. Instead of blocking it, the server inserts a record into the 'userIdentities' table to link GitHub as a provider to the existing user and signs a new JWT cookie.
4. The user is redirected to the dashboard, confirming a new active session.

Case 5: Existing Local User Logs In Successfully

1. Returning users type in their email and password and click **Sign In**.
2. The server uses bcrypt to check the password against the stored hash, finds a match, and creates a new JWT.
3. It stores the token as a secure cookie and updates the last-login time and source.
4. The user is redirected to the dashboard, confirming a new active session.

Case 6: Existing Local User Enters Wrong Password

1. The user types a familiar email but makes a typo in the password.
2. The server detects the mismatch and sends back “Invalid credentials” without issuing any cookie.
3. The frontend displays a generic and ambiguous message, such as “Incorrect credentials. Please check your email and password.”
4. For improved UX, the frontend pre-fills the email field, allowing the user to focus on correcting their password.
5. From the user’s perspective, nothing crashes, just a friendly reminder to try again.

Case 7: Returning OAuth User Logs In Again

1. An existing Google user comes back after a few days and clicks the same “Sign in with Google.”
2. Google sends the authorization code, our server verifies it, finds the email in the database, refreshes their JWT cookie, and updates the `lastLoginAt` in the `users` table.
3. The user is redirected to the dashboard, confirming a new active session.

Case 8: OAuth Provider Returns No Email (GitHub Edge Case)

1. A GitHub user approves login, but their email is private.
2. Our server calls a second GitHub API to fetch verified emails.
3. If none are available, the system rejects the login and informs the user to either set their email to public on the OAuth provider or use another sign-in method.
4. The frontend displays an error message explaining the requirement for a verified email address.
5. No user record is created, and the user is not logged in.

Case 9: Duplicate Local Signup Attempt

1. A user attempts to register on the **Sign Up page** using an email that’s already in the database.
2. The server immediately finds the record and responds with “User already exists”.
3. Instead of an error page, the user sees a suggestion to sign in or use Google/GitHub.
4. It feels helpful rather than technical, and no duplicate user is created.

Case 10: Existing OAuth User Adds Another Provider

1. A user who usually logs in with Google now tries GitHub.
2. GitHub returns the same verified email, so our server links this new provider to the same account.
3. GitHub returns the same verified email, so our server links this new provider to the same account by adding the new provider's ID to the existing user's record in the `'userIdentities'` table and issuing a JWT.
4. The user is redirected to the dashboard, confirming a new active session.

Case 11: Suspended or Inactive User Tries to Log In

1. An old account marked “suspended” tries to sign in.
2. The backend detects the status flag before any password check and stops the login.
3. It sends a “403 Account suspended” message and logs the event for audit.
4. The screen shows a gentle notice: “Your account is suspended. Contact support.”

5. No access is granted, and the session never starts.

Case 12: OAuth Callback with Invalid State or Expired Token

1. A user approves Google login but takes too long to return.
2. The temporary token expires.
3. When they come back, the backend compares the state and finds it invalid.
4. It rejects the request, logs it as a security incident, and shows “Login failed, please try again.”
5. The user simply starts over—no risk, no data leak.

Case 13: Expired or Missing JWT Cookie

1. A user tries to open a protected page after hours of inactivity.
2. The middleware checks their cookie and finds it expired.
3. The server clears the cookie and redirects to the login page, appending a query parameter (`?reason=expired`) specifically when the token is found to be expired.
4. The frontend checks for this query parameter to display the specific message: “Your session has expired, please sign in again.”
5. If the cookie was simply missing rather than expired, the redirect to the login page remains silent to keep the security flow seamless for the user.

Case 14: Logout Flow

1. A logged-in user clicks “Logout.”
2. The frontend sends a logout request to the server, which immediately clears the JWT cookie and returns success.
3. The browser redirects to the home page.
4. The user sees the public landing screen again session completely ended.

Case 15: Same Email Used for Local and OAuth Login

Case 16: User Logs In Again After Logout

1. The backend finds a match and links the new OAuth identity to the existing user record by inserting a record into the `userIdentities` table.
2. Their roles and permissions stay the same.
3. The user is redirected to the dashboard, confirming a new active session.

Case 16: User Logs In Again After Logout

Case 17: Invalid or Mistyped Email

Case 18: OAuth Exchange Fails (Network or Server Error)

1. The user is redirected to the dashboard, confirming a new active session.

Case 17: Invalid or Mistyped Email

1. Someone types “meghana@@gmail” by mistake.
2. The frontend instantly flags it as invalid and prevents submission.
3. If they somehow bypass that, the backend double-checks and returns “400 Invalid email.”
4. The screen politely asks them to enter a valid address before continuing.
5. No server load, no confusion—just helpful feedback.

Case 18: OAuth Exchange Fails (Network or Server Error)

1. During login with Google, a temporary network issue interrupts the token exchange.
2. The server times out, logs the error, and returns a message saying, “Login temporarily unavailable. Please try again later.”
3. The page shows a simple error notice, and no account is created.
4. The system remains secure and stable even when something goes wrong.

6.5.4. Validation and Mismatch Tests

These tests verify that the backend correctly rejects invalid input and security protocol errors before granting a session, maintaining integrity across both local and OAuth paths.

- **Invalid Email Syntax (Local Signup/Login):** User attempts to submit an email that fails basic RFC validation (e.g., “test@@email.com”). → **400 INVALID_EMAIL**
- **Missing Required Fields (Local Signup/Login):** User omits the password field on `/api/auth/signin` or `/api/auth/signup`. → **400 MISSING_FIELD**
- **Password Complexity (Local Signup):** User submits a password that fails minimum complexity or length requirements. → **400 PASSWORD_WEAK**
- **Duplicate Local Signup Attempt:** User attempts to register on `/api/auth/signup` with an email that already exists in the `users` table. → **409 EMAIL_EXISTS**
- **OAuth State Mismatch (CSRF):** The `state` parameter returned by the OAuth provider does not match the stored state on the server. → **401 INVALID_STATE**
- **OAuth Exchange Failure:** The server fails to exchange the authorization code for a provider token (e.g., provider error, expired code). → **503 OAUTH_EXCHANGE_FAILED**

- **Invalid Token Signature:** An attacker attempts to submit a manually modified JWT cookie to a protected endpoint. → 401 UNAUTHORIZED
- **Expired Token in Protected Route:** An expired JWT is used to access a protected route. → 401 UNAUTHORIZED
- **Insufficient Role for Route:** An authenticated user accesses a route that requires a higher role than they possess (e.g., 'viewer' tries to access an 'admin' endpoint). → 403 FORBIDDEN

Each validation ensures the backend rejects malformed input *before* issuing any token, maintaining integrity across OAuth and local paths.

6.5.5. End-to-End Tests

End-to-end (E2E) tests confirm that all components of the upgraded architecture—including the React frontend, authentication middleware, secure HTTP-only cookies, and database identity linking—work together seamlessly across the full user journey. These tests validate the system from the user's perspective, ensuring that transitions between different states and providers are handled correctly and securely.

- **Identity Unification:** A user registers locally and later logs in with OAuth using the same email; the system successfully links to the single existing account and verifies the `userIdentities` table update.
- **Dual-Track Coexistence:** A user successfully logs in with a local password, logs out, and then immediately logs in with Google (or vice versa), demonstrating clean cookie issuance across both authentication paths.
- **Full Role Enforcement:** An authenticated “viewer” user is blocked with a 403 Forbidden when accessing a specific protected Admin API endpoint (e.g., `/api/admin/config`), while an “admin” user is granted access.
- **Secure Session Termination:** After a successful `POST /api/auth/logout`, subsequent requests to protected routes (e.g., `/api/auth/check`) return a 401 Unauthorized, confirming the secure cookie has been completely cleared.
- **New User Onboarding:** A brand new user completes registration via OAuth and is correctly issued a session cookie and redirected to the profile edit page to complete required fields.

6.5.6. Monitoring and Telemetry

This section defines the operational playbook for post-deployment health validation. It outlines the specific, measurable metrics and thresholds that must be instrumented and monitored in production to confirm the system is secure, stable, and performing efficiently. It is the final step that transitions validation from a QA checklist to a real-time SRE requirement.

Metric	Target	Purpose
--------	--------	---------

P95 Auth Latency	< 50 ms	Measures the P95 latency of the requireAuth middleware to confirm fast-path validation efficiency.
Unexpected 401/403 Rate	< 0.01%	Tracks security incidents; a spike indicates token tampering or middleware bugs (excluding intentional token expiry).
OAuth Callback Failure Rate	< 0.5%	Measures health of the new openid-client integration and external provider dependency stability.
Failed Identity Merges	= 0	Critical Data Integrity Check. Ensures the account linking logic for the userIdentities table is working perfectly.
Login Distribution	Monitor Only	Tracks user adoption of Local, Google, and GitHub sign-in methods.

Logs capture every `/login`, `/logout`, and `/auth/callback` event with timestamps and status codes for audit trails. Alerts are configured to trigger immediately on any Failed Identity Merge (critical P1 alert), and if the P95 Auth Latency exceeds 150 ms for five consecutive minutes (performance alert). A spike in the Unexpected 401/403 Rate or OAuth Callback Failure Rate ($> 5\%$) will trigger a security/health alert.

7. Risk Management and Fallback Plan

Risk management anticipates technical and operational failures before they affect users. Fallback procedures define how to restore service without data loss or downtime.

7.1. Primary Risks and Mitigations

Risk	Description	Mitigation / Fallback
OAuth Provider Outage	Google/GitHub API unreachable	Keep local login route active; cache provider metadata; gracefully notify user
Cookie Misconfiguration	Wrong domain or flags block sessions	Validate cookie after deploy using staging tests; auto-fallback to legacy JWT header for 24 h

Account Link Conflicts	Same email used by two providers	Admin resolution tool to merge records and audit activity
CSRF or State Mismatch	OAuth state tampered	Immediate INVALID_STATE error + log event for review
Rate Limit False Positives	Aggressive limiter blocks legit users	Adaptive thresholds and whitelist internal IPs
Schema Migration Failure	ALTER TABLE rollback risk	Run dry-run migration on shadow DB; snapshot backup before live change

If critical errors occur, disable OAuth routes, clear cookies, and temporarily restore the previous local JWT flow.

No user data is deleted; re-enabling OAuth is a one-line config change.

8. Verification and Success Criteria

Verification translates design goals into observable facts. Success criteria prove that each component operates exactly as intended after deployment.

8.1. Operational Verification

- Users can log in with Google, GitHub, or local credentials and reach their dashboard within < 3 seconds.
- JWT cookies never appear in browser storage or network responses in plain text.
- Role-based authorization continues to filter restricted routes.
- Tampered or expired cookies return **401 UNAUTHORIZED**.
- Frontend navigation reflects permissions accurately (admin vs viewer).
- Session cookies expire and refresh according to TTL policy (two hours default, sliding on activity).
- Monitoring dashboards show < 1% auth error rate after launch.
- Database integrity check confirms unique (provider, id) and (email) constraints.

8.2. User Level Validation

- First-time OAuth users are prompted to complete profile fields once.
- Linked accounts preserve existing roles and content.
- Logout fully invalidates the session and redirects home.

Meeting all criteria signals readiness for production.

9. Completed Stretch Goals

This section documents the features originally classified as future enhancements (see Appendix D) that were implemented and deployed as part of this upgrade cycle. Each subsection describes what was built, how it works, and the implementation decisions made during development.

9.1. Password Reset Functionality

Users who forget their password — or who created their account via OAuth and never set one — need a secure, self-service way to establish local login credentials. This feature implements the industry-standard forgot-password flow using a time-limited, single-use token delivered by email.

9.1.1. User Flow

1. The user clicks "**Forgot password?**" on the Sign In page and is taken to the Forgot Password page.
2. They enter their email address and submit.
3. The server looks up the email. If found, it generates a secure random token, stores it with an expiry, and sends a reset email. If not found, it returns the same success message to prevent email enumeration.
4. The user clicks the link in the email and is taken to the Reset Password page.
5. They enter and confirm a new password.
6. The server validates the token, hashes the new password, updates the `users` table, and invalidates the token.
7. The user is redirected to Sign In with a success message.

9.1.2. Database Changes

SQL

```
-- Stores time-limited password reset tokens
CREATE TABLE passwordResetTokens (
  token      VARCHAR(64) NOT NULL,
  userID     INT          NOT NULL,
  expiresAt  DATETIME     NOT NULL,
  PRIMARY KEY (token),
  FOREIGN KEY (userID) REFERENCES users(userID) ON DELETE CASCADE
);
```


9.1.3. Backend Routes

Endpoint	Behavior
POST <code>/api/auth/forgotpassword</code>	Lookup email → generate token → store with 1-hour expiry → send reset email → return success regardless of whether email exists
POST <code>/api/auth/reset-password</code>	Validate token exists and not expired → hash new password → update <code>users</code> table → delete token → return success

9.1.4. Email Transport

A shared `utils/mailer.js` module handles transport selection:

- **Development** (`NODE_ENV !== 'production'`): Ethereum fake SMTP; logs preview URL to console
- **Production**: Local sendmail via Postfix; requires `NODE_ENV='production'` in the server `.env`

All outgoing mail is sent from `noreply@driftconditions.org`. Both HTML and plaintext versions are included in every message.

Important: `NODE_ENV=production` must be explicitly set in the server environment. If omitted, the mailer silently falls back to Ethereum and no email reaches the user.

9.1.5. Reset Email Design

- Subject: "Reset your DriftConditions password"
- Body: "We received a request to reset the password for your DriftConditions account. Click the button below to choose a new password."
- Expiry notice: "This link expires in 1 hour."
- Safety notice: "If you didn't request a password reset, you can safely ignore this email. Your password will not be changed."

9.1.6. Frontend Pages

- **ForgotPassword.js** — email input form; shows generic success message after submit regardless of outcome
- **ResetPassword.js** — reads token from URL query param; new password + confirm fields; submits to `/api/auth/reset-password`
- **Signin.js** — adds "Forgot password?" link below the form

9.1.7. Security Considerations

- Tokens are 32-byte cryptographically random hex strings (`crypto.randomBytes(32)`)
- Tokens expire after 1 hour
- Tokens are single-use – deleted immediately upon successful password reset
- The forgot-password endpoint returns identical responses for found and not-found emails to prevent account enumeration
- Rate limiting applied to both endpoints via the existing `authLimiter`

9.2. User Avatars from OAuth Providers

OAuth providers supply a profile avatar URL at login. The system captures it on every OAuth login and displays it on the profile page, falling back to a placeholder icon if none is set.

9.2.1. Database Schema

SQL

```
ALTER TABLE users ADD COLUMN avatar_url VARCHAR(512) DEFAULT NULL;
```

9.2.2. Server: Capturing Avatar URLs

Each OAuth provider exposes an avatar through a different field. During the OAuth callback, the avatar URL is extracted alongside other profile fields:

Provider	Avatar Field
Google	<code>claims.picture</code> (from OIDC token claims)
GitHub	<code>ghUser.avatar_url</code> (from <code>/user</code> API response)
Discord	Constructed: <code>https://cdn.discordapp.com/avatars/{id}/{avatar}.png</code>

For Discord, if `discordUser.avatar` is null (user has no custom avatar), `NULL` is stored.

9.2.3. Update Strategy

`avatar_url` is updated on **every OAuth login** ("last login wins"). The most recently used provider's avatar is always shown. This reflects the user's implicit preference – whichever provider they log in with most recently is the one whose avatar appears.

- For new users: `avatar_url` is included in the initial `INSERT`
- For returning users: updated via `UPDATE users SET avatar_url = ? WHERE userID = ?` at the end of the OAuth callback

9.2.4. Client: Display Logic

The profile page checks for `avatar_url` in the returned user data:

- If present: renders `<img src={avatarUrl} alt="avatar" referrerPolicy="no-referrer" />` inside the avatar container
- If absent: falls back to the existing FeatherIcon `user` silhouette

The `referrerPolicy="no-referrer"` attribute is required to load Google-hosted avatar images, which are blocked without it. Avatar URLs are used directly — no proxying or server-side caching.

Avatar images are displayed in a circular frame using `object-fit: cover` so they fill the container regardless of the source image's aspect ratio.

9.3. Discord OAuth Support

The original plan specified Google and GitHub as OAuth providers. Discord was added as a third provider during implementation. The integration follows the same pattern:

- **Initiation:** `GET /api/auth/discord` redirects to Discord's OAuth consent screen
- **Callback:** `GET /api/auth/callback/discord` exchanges the code for a profile
- **Email handling:** Discord accounts may have unverified emails; the system requires a verified email and rejects logins where none is available
- **Avatar:** Constructed from the user's Discord ID and avatar hash (see 8.2.2)
- **Callback URL:** Because Discord allows only one redirect URI per OAuth app, a separate Discord application is required for production vs. development

9.4. Public Page Auth Persistence

After the OAuth integration was deployed, a regression was discovered: users who navigated to public pages (homepage, How It Works) after logging in would lose their session state on reload, even though their cookie was valid.

9.4.1. Root Cause

The original `useAuthCheckAndNavigate` hook skipped the `POST /api/auth/check` call entirely for public pages, on the assumption that unauthenticated access should be allowed without any check. This meant the Redux auth store was never populated for public pages — so on reload, the app had no record of the logged-in user.

9.4.2. Fix: Always Check, Only Redirect on Failure

The hook was refactored to always await the auth check, but only redirect on failure for protected pages:

JavaScript

```
const noAuthPages = ['error', 'notauth', 'homepage', 'signup', 'signin',
  'forgotpassword', 'resetpassword', 'howitworks'];

const isPublicPage = noAuthPages.includes(context.toLowerCase());

// Always run the check to populate user state
const actionResult = await dispatch(checkPageAuth({ context }));
const result = actionResult.payload;

if (result && result.error) {
  // Only redirect on auth failure for protected pages
  if (!isPublicPage) {
    if (result.error.reason === "not_authenticated") navigate('/signin');
    else if (result.error.reason === "not_authorized")
      navigate('/notauth');
  }
}
```

9.4.3. Backend: Public Context Bypass

Running the auth check for public pages introduced a side effect: the backend's permission check returned a 403 for unauthenticated users visiting the homepage, which broke the playlist component. The fix was a `publicContexts` allowlist in the check route that skips the permission check for known public pages:

JavaScript

```
const publicContexts = ['homepage', 'signin', 'signup', 'notauth',
  'error', 'howitworks', 'forgotpassword', 'resetpassword'];

if (!publicContexts.includes(pageContext) &&
  !user.permissions.includes(pageContext)) {
  return res.status(403).json({ ... });
}
```

9.5. Environment Configuration Templates

To support repeatable deployment across development and production environments, two documented template files were added to the `setupfiles/` directory:

- `setupfiles/env.local` — template for local development; uses Ethereum email, `localhost` callback URLs, and a single GitHub OAuth app
- `setupfiles/env.server` — template for the production server; uses Postfix email, `driftconditions.org` callback URLs, and a separate GitHub OAuth app (GitHub allows only one callback URL per app)

Each file is fully commented, documents where credentials are obtained, and explains the behavioral differences between `NODE_ENV=development` and `NODE_ENV=production`. Neither file is committed with real secrets; the actual `.env` files remain gitignored

10. Appendices

10.1. Appendix A — Database Schema Diff

See Section 5.2.1.

10.2. Appendix B — OAuth Flow Diagram

User → Provider → Backend → Cookie → `/auth/check` → Authenticated Session
(See Section 3.2 for full step sequence.)

10.3. Appendix C — Test Matrix Summary

Derived from DriftConditions Auth Testcases document:

- 12 major flows verified (local auth × 5, OAuth × 2, merge × 2, session/logout × 3).
- Covers duplicate prevention, automatic merge, and edge session cases.
- Each test validated against expected HTTP codes and state transitions.

10.4. Appendix D — Future Enhancements and Technical Debt

10.4.1. JWT Secret Rotation (Version 2)

While the current design validates JWTs, the secret key is currently fixed. For long-term security, implementing a **Multi-Key Rotation** strategy is required to allow zero-downtime rotation of the JWT signing secret. This involves the backend managing two or more active secrets: a primary secret for signing new tokens, and secondary secrets for verifying existing, still-valid tokens.

1. Database Changes (Recommended)

To ensure persistence and consistency across backend instances, the active secrets should be stored in the database:

- **Create a new table** (`jwtSecrets`) to store the active keys and their version IDs.

Column	Type	Purpose
<code>kid</code>	INT PRIMARY KEY	Key ID (version number)
<code>secretKey</code>	VARCHAR(255)	The actual secret key value (must be highly secured)
<code>isPrimary</code>	BOOLEAN	<code>TRUE</code> if used for signing new tokens
<code>createdAt</code>	DATETIME	Timestamp for rotation tracking

Note: Make sure these schema fields correspond to existing schema schemes.

2. Backend Logic Changes (Node/Express)

The `requireAuth` middleware and token issuance utility must be updated to handle multiple keys and leverage the `kid` header for versioning:

- **Token Issuance:** The backend must use the `secretKey` marked as `isPrimary=TRUE` for signing and include the corresponding **Key ID (`kid`)** in the JWT header.
- **Token Verification:** The `requireAuth` middleware will read the `kid` from the JWT header, retrieve the corresponding `secretKey` from the `jwtSecrets` table, and attempt verification.

3. Rotation Process

Rotation should be managed by a scheduled job:

- **Generate New Key:** Create a new random secret and assign a unique `kid`.
- **Promote:** Insert the new key into `jwtSecrets` and set it as `isPrimary=TRUE`. The old primary key is automatically demoted to a secondary (verification-only) key.
- **Cleanup:** After the longest possible token lifetime (e.g., 2 hours plus safety margin), delete old, inactive keys from the table.

10.4.2. Mandatory Email Verification for Local Sign-up (Zombie Account Mitigation)

To eliminate the risk of "zombie accounts" (unverified, unusable user records created due to mistyped but syntactically valid emails), we will introduce mandatory email verification for all local sign-ups. This requires changing the current POST `/api/auth/signup` flow (Case 1) to a two-step process:

1. **POST /api/auth/signup**: Server creates the user record with a status of `isVerified = FALSE`, but does not issue a JWT session cookie.
2. **Verification Email**: The server sends a time-limited verification link to the provided email address.
3. **GET /api/auth/verify**: Only upon the user clicking the link does the backend update `isVerified = TRUE` and issue the session JWT cookie.

Any unverified user accounts will be automatically purged after a 24-hour timeout. This prevents database bloat and ensures that only contactable users are issued sessions.

This approach ensures tokens issued before a rotation can still be verified for their full lifespan.

10.4.3. Server-Side Session Revocation (JWT Blacklist)

To address the lack of server-side session control (Section 2.3), a future enhancement will implement a JWT Blacklist (Revocation List) using a fast, centralized cache like Redis. When a user logs out (Case 14) or an admin revokes a token, the token's ID (JTI) will be added to this Blacklist. The `requireAuth` middleware will then check this Blacklist before validating the token's signature, ensuring immediate invalidation of compromised or terminated sessions.

10.4.4. Other Possibilities

High value, low lift:

- **Email verification on signup** — same infrastructure, send a verification link on register

Medium lift, natural extensions:

- **"Set a password" for OAuth-only users** — they have a hex placeholder; need a flow that lets them set one without knowing the current password (since there isn't one)
- **Manage linked providers** — show which OAuth providers are connected to an account, allow linking/unlinking from `ProfileEdit`

Nice to have:

- **Login history / last seen** — `lastLoginAt` is already being written; just needs a display somewhere

10.5. Appendix E — Document Style Guide

This section details the standardized formatting applied to the document.

10.5.1. Headers and Section Titles

This style dictates the formatting for all section titles and headings (Heading 1, Heading 2, Heading 3, etc.) to ensure clear hierarchy and visual distinction from body text. All headers must

be formatted exclusively using these standard Google Docs heading styles rather than custom text styles.

- **Font Family:** Arial
- **Font Size:** Varied (e.g., 20pt for Heading 1, 16pt for Heading 2, 14pt for Heading 3)
- **Font Weight:** Bold
- **Application:** Applied exclusively to section titles and subtitles, using the designated Docs Heading styles.
- **Structural Patterns:** H1 headings (e.g., "1. Goals and Scope") must use the "Heading 1" doc style; H2 headings (e.g., "1.1. Goals") must use the "Heading 2" doc style; and this hierarchical pattern must be followed for all subsequent heading levels.

10.5.2. Body Paragraphs and Standard Prose

This section defines the baseline formatting for all narrative and descriptive text in the document. Standardized prose ensures optimal readability and consistency across all sections, distinguishing it clearly from headings and technical lists.

- **Font Family:** Arial
- **Font Size:** 11pt
- **Line Spacing:** 1.15
- **Spacing (Before Paragraph):** 0pt
- **Spacing (After Paragraph):** 12pt
- **Application:** Applied exclusively to normal body text, excluding headings, titles, and list items.

10.5.3. Inline Technical References

This style is used for all short code references embedded within the main text, such as variable names, file paths, API routes, and terminal commands. It serves to visually isolate code elements from surrounding narrative text.

- **Font Family:** Roboto Mono (Monospaced)
- **Text Color:** Gemini Green (#188038)
- **Background:** None
- **Use Cases:** Inline code snippets (e.g., `clipUsage` table), variable names (e.g., `audioID`), file paths (e.g., `config/config.js`), and console output text.

10.5.4. Code Block Syntax Highlighting Style: One Light (Notion variant)

A light-background syntax theme derived from the One Light family, originally developed for the Atom editor. Clean, low-contrast, and readable on white or near-white backgrounds.

Background: Light gray (#f7f7f7 or similar off-white) **Default text:** Dark gray (identifiers, punctuation, table/column names)

Token colors:

- **Keywords** (`ALTER`, `CREATE`, `ADD`, `NOT`, `NULL`, `ON`, `DELETE`): Medium blue (`#0070c1` or similar)
- **Types** (`INT`, `VARCHAR`, `DATETIME`): Bright blue (`#0099cc` or similar)
- **Numeric literals** (`255`, `32`, `191`): Orange-red (`#e06c00` or similar)
- **Comments** (`-- ...`): Muted pink/mauve (`#c678a0` or similar)
- **Language label** (`SQL`): Small-caps, light gray, non-highlighted

Typography: 10pt, Monospace, medium weight, single line height. Token colors are distinct but not garish — optimized for readability over visual drama.

```
SQL
-- Existing 'users' table update (only adding utility columns)
ALTER TABLE users
  ADD COLUMN displayName VARCHAR(255),
  ADD COLUMN lastLoginAt DATETIME;

-- New 'userIdentities' table to support multiple linked OAuth accounts
CREATE TABLE userIdentities (
  userId INT NOT NULL, -- Foreign key to the users table
  oauthProvider VARCHAR(32) NOT NULL,
  oauthId VARCHAR(191) NOT NULL,
  PRIMARY KEY (oauthProvider, oauthId),
  FOREIGN KEY (userId) REFERENCES users(userID) ON DELETE CASCADE
);

-- Allow null names since OAuth providers may not supply them
ALTER TABLE users
```